MATRIXPOLICY: ROLE-AWARE OPTIMIZATION FOR RATIONAL TRANSFORMER NONLINEARITIES

Anonymous authors
Paper under double-blind review

ABSTRACT

Transformer nonlinear sublayers contain an input matrix, a coordinatewise nonlinearity, and an output matrix, but standard optimizers usually update these matrices without using their distinct functional roles. We introduce a global-rational Rational Latent Basis (RLB) sublayer that exposes normalized group coordinates, trainable real-pole-free P5/Q4 responses, derivative/output response summaries, and an idealized positive matrix-pair rescaling relation that becomes approximate under the implemented RMS floor. We then introduce MatrixPolicy, a role-aware optimizer for the RLB input and output matrices. In the reported configuration, rational coefficients and all non-RLB-matrix Transformer parameters use AdamW, while MatrixPolicy applies centered group-gradient redistribution, a transient gated Muon substep, and bounded matrix-pair rescaling only to the RLB matrices. The main empirical benefit is faster arrival at useful loss levels, measured primarily as the percentage of comparator target-arrival tokens saved. Across five 12-layer, width-768 language-model pretraining corpora, MatrixPolicy saves 19.5–22.8% versus SiLU with AdamW and 29.2–34.0% versus SiLU with Muon under the 100M-token budget; under the 300M-token budget it saves 22.3–26.4% versus SiLU with AdamW and 6.7–9.8% versus SiLU with Muon, with lower measured target-arrival time for every dataset/comparator pair. TODO: Test whether these percentage target-arrival savings persist in a larger-scale experiment beyond the current fixed-scale setting.

1 INTRODUCTION

Transformer nonlinear sublayers are usually described by their scalar activation: GELU, SiLU, SwiGLU, or a related gated variant (Hendrycks & Gimpel, 2016; Ramachandran et al., 2017; Shazeer, 2020). That shorthand hides a larger optimization object. A sublayer first chooses coordinates through an input matrix, applies a nonlinear response in those coordinates, and then recombines the resulting features through an output matrix. The two matrices do not play the same role, yet AdamW, Muon, and other general optimizers usually see them as ordinary tensors with no knowledge of which side of the nonlinearity they occupy.

We study whether the nonlinear sublayer can expose enough structure for the optimizer to use. The key idea is to make the activation an interface, not only a pointwise function. If the sublayer provides normalized groups, a trainable rational response in each group, observable derivative and curve-response scales, and a controlled positive rescaling relation between the adjacent matrices, then the optimizer can assign different update behavior to the input and output roles without changing the rest of the Transformer.

We instantiate this idea with a global-rational Rational Latent Basis (RLB) sublayer. It projects the residual stream into grouped latent coordinates, RMS-normalizes each group, applies one trainable real-pole-free P5/Q4 response per group, and projects back to the model width. The main variant used in this paper has no active local atoms: there are no trainable atom centers and no atom coefficient parameters. This choice keeps the nonlinear response simple enough to audit while preserving the matrix-role signals that MatrixPolicy uses.

MatrixPolicy is the optimizer coupled to this sublayer. It is not a generic optimizer for every Transformer tensor. It acts only on the two RLB matrices, using batch-estimated rational gain proxies, relative gradient scales, and the matrix pair ratio to redistribute group updates, apply a transient

matrix-direction substep, and keep the input/output representatives balanced. In the reported configuration, rational coefficients, attention weights, embeddings, and all other non-RLB-matrix parameters use AdamW.

We evaluate the method on a fixed 12-layer, width-768 causal Transformer across DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4. The 100M-token budget captures early optimization differences before long-budget saturation compresses endpoints. In this regime MatrixPolicy reaches common validation-loss targets substantially earlier than AdamW and Muon comparators on both SiLU and global-rational RLB controls. The 300M-token budget extends the same architecture to a longer run; endpoint gaps are smaller after longer training, but MatrixPolicy still keeps lower endpoint validation loss across all five datasets.

The contributions are:

- a global-rational RLB sublayer that exposes matrix roles, curve-gain summaries, and an idealized positive matrix-pair relation without local atom parameters;
- MatrixPolicy, a role-aware optimizer for the global-rational RLB input and output matrices in a reported configuration where rational coefficients and non-RLB-matrix parameters use AdamW;
- a matched empirical package showing faster 100M-token-budget target arrival in both tokens and measured clean target-arrival time against AdamW and Muon controls, with 300M-token-budget saturation results showing that the endpoint advantage persists after longer training.

2 Related Work

Transformer nonlinear sublayers are often identified by their scalar response, from GELU and Swish/SiLU to gated variants such as GLU and SwiGLU (Hendrycks & Gimpel, 2016; Ramachandran et al., 2017; Shazeer, 2020). RLB follows the same architectural location but changes the interface exposed to optimization: instead of replacing only a fixed scalar curve, it exposes grouped coordinates, trainable rational responses, and the adjacent matrix roles used by MatrixPolicy.

Rational activations are motivated by the ability of low-degree rational functions to represent sharp transitions, saturation, and curved response shapes with few parameters. Recent approximation-theoretic work argues that trainable low-degree rational activations can have expressivity advantages over common fixed activations (Tang & Townsend, 2026). We use that result as motivation for the activation family, not as a proof of MatrixPolicy. The empirical comparison separates the global-rational RLB sublayer from the role-aware optimizer by including RLB controls with AdamW, Muon, Lion, SOAP, CAME, and ScheduleFree-style optimizers.

Adam and AdamW remain standard Transformer pretraining optimizers (Kingma & Ba, 2015; Loshchilov & Hutter, 2019). Other work changes the first-order rule while keeping a generic tensor interface, including Lion and cautious optimizers (Chen et al., 2023; Liang et al., 2026), or uses matrix structure and low-rank information, including Muon, Sophia, GaLore, SOAP, and Adam-mini (Liu et al., 2025; 2024; Zhao et al., 2024; Vyas et al., 2025; Zhang et al., 2025). CAME and schedule-free training modify optimizer state or schedules (Luo et al., 2023; Defazio et al., 2024). MatrixPolicy is closest in spirit to matrix-aware optimization, but in the reported configuration it applies role-specific rules only to the input and output matrices of RLB; rational coefficients and non-RLB-matrix Transformer parameters use AdamW. Broader optimizer coverage is available in recent surveys (Wen et al., 2026).

3 Method

RLB and MatrixPolicy form an activation–optimizer pair: RLB exposes detached sublayer summaries, and MatrixPolicy uses them to update only the adjacent input/output matrices.

Throughout the method we use

$$\mathrm{clip}(x, a, b) = \min\{\max\{x, a\}, b\}, \tag{1}$$

$$\mathrm{smoothstep}(a, b, x) = 3\omega^2 - 2\omega^3, \qquad \omega = \mathrm{clip}\left(\frac{x-a}{b-a}, 0, 1\right). \tag{2}$$

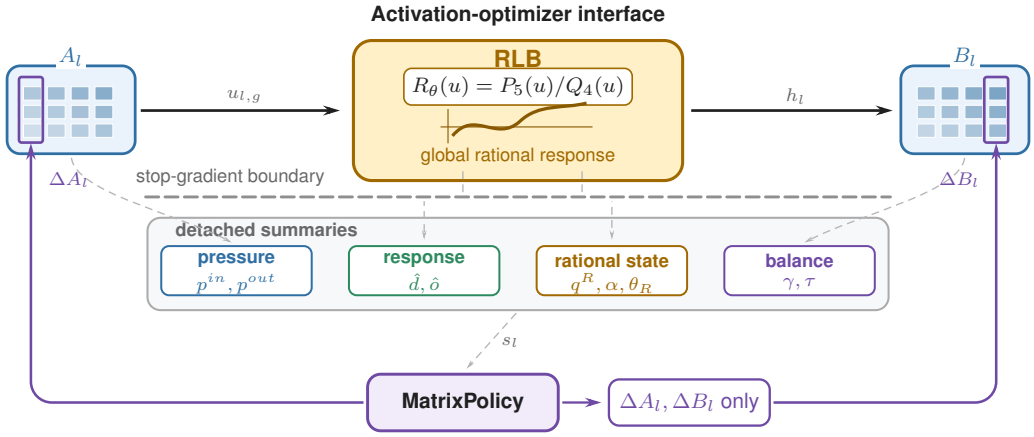

Figure 1: Activation–optimizer interface for global-rational RLB. In layer l, A_l maps the sublayer input into normalized rational groups and B_l recombines the rational features. RLB exposes matrix-pressure, response-gain, observed-rational-state, and pair-balance summaries through a stop-gradient boundary. MatrixPolicy reads the resulting summary vector s_l and writes only ΔA_l and ΔB_l; rational coefficients are AdamW-trained and policy-observed, not MatrixPolicy update targets.

The interface in Figure 1 is formalized by the RLB sublayer below and the MatrixPolicy update in Section 3.3.

3.1 Rational Latent Basis Sublayer

For Transformer layer $l \in \{1, \ldots, L\}$, let $x_l \in \mathbb{R}^d$ denote the input to the nonlinear sublayer. RLB first projects this input to a latent width $H = Gm$ and partitions the latent vector into G groups of width m:

$$z_l = A_l x_l, \qquad z_l = \mathrm{concat}_{g=1}^{G} z_{l,g}, \qquad z_{l,g} \in \mathbb{R}^m, \tag{3}$$

where $A_l \in \mathbb{R}^{H \times d}$ is the input-domain matrix. Each group is normalized by its root-mean-square radius,

$$r_{l,g} = \sqrt{m^{-1}\|z_{l,g}\|_2^2 + \epsilon_{\mathrm{rms}}}, \qquad u_{l,g} = z_{l,g}/r_{l,g}, \tag{4}$$

with numerical floor $\epsilon_{\mathrm{rms}} > 0$. A scalar response is applied coordinatewise in the normalized coordinates and then multiplied by the same radius:

$$h_{l,g} = r_{l,g} R_{l,g}(u_{l,g}), \qquad h_l = \mathrm{concat}_{g=1}^{G} h_{l,g}, \qquad y_l = B_l h_l, \tag{5}$$

where $B_l \in \mathbb{R}^{d \times H}$ is the output-recombination matrix.

The curve $R_{l,g}$ is a trainable real-pole-free piecewise-rational P5/Q4 response,

$$R_{l,g}(u) = \frac{P_{l,g}(u)}{Q_{l,g}(u)}, \qquad P_{l,g}(u) = \sum_{i=0}^{5} a_{l,g,i} u^i, \tag{6}$$

with denominator

$$Q_{l,g}(u) = 1 + |b_{l,g,1}|\,|u| + |b_{l,g,2}|\,u^2 + |b_{l,g,3}|\,|u|^3 + |b_{l,g,4}|\,u^4. \tag{7}$$

The absolute-value terms make the response piecewise rational rather than a single algebraic rational function of u, but they guarantee $Q_{l,g}(u) \geq 1$ on the real line. Derivative-gain statistics use the autodiff derivative of the implemented expression, with $\operatorname{sign}(0) = 0$; equations involving $R'_{l,g}$ are understood as almost-everywhere derivatives. Numerator and denominator coefficients are initialized by fitting SiLU on $[-5, 5]$ and are trained thereafter by AdamW in the reported configuration; MatrixPolicy only observes their gradients. The implementation uses the same interval for probe-grid gain summaries. In the main variant there are no active local Cauchy atoms, trainable atom centers, or atom coefficient parameters.

The normalization in equation 4 gives the optimizer two views of each group. The normalized coordinate $u_{l,g}$ determines where the response is evaluated, while the radius $r_{l,g}$ carries group scale back to the residual stream. For fixed curves $R_l = \{R_{l,g}\}_{g=1}^{G}$, define the RLB block map

$$f_l(x; A_l, B_l, R_l) = B_l \operatorname{concat}_{g=1}^{G} r_{l,g} R_{l,g}(u_{l,g}), \tag{8}$$

where $z_l = A_l x$, $z_{l,g}$, $r_{l,g}$, and $u_{l,g}$ are computed by equation 3–equation 4. In the idealized case $\epsilon_{\mathrm{rms}} = 0$ with nonzero group radii, this construction admits the positive rescaling

$$f_l(x; A_l, B_l, R_l) = f_l(x; D_l(a) A_l, B_l D_l(a)^{-1}, R_l), \tag{9}$$

where $D_l(a) = \operatorname{diag}(a_1 I_m, \ldots, a_G I_m)$ for $a \in \mathbb{R}^G_{>0}$. MatrixPolicy uses this relation only for small, bounded balancing steps because the additive RMS floor makes the block-map identity approximate, and finite optimizer-state handling makes subsequent dynamics only approximately invariant in the implemented system.

3.2 Role-Aware Optimizer Statistics

MatrixPolicy tracks role-specific relative gradient scales and curve-gain summaries for each layer and group. At training step t, the gradient-based statistics in this subsection are computed after backpropagation on the current minibatch and before any parameter update, including AdamW, Muon, or pair rescaling. The resulting values update the moving averages used by the same optimizer step unless otherwise noted. For any tensor V, define

$$\operatorname{rms}(V) = \sqrt{\operatorname{mean}(V^2)}, \qquad \operatorname{rms}_\epsilon(V) = \sqrt{\operatorname{mean}(V^2) + \epsilon}. \tag{10}$$

The constants $\epsilon_p > 0$ and $\epsilon_g > 0$ denote fixed additive floors in squared-RMS units. The same ϵ_p is reused as the implementation floor for matrix-ratio logs in equation 29. The resulting pressure and activity scores are fixed-implementation control signals rather than reparameterization-invariant scalar observables. Let $A_{l,g} \in \mathbb{R}^{m \times d}$ be the row block of A_l, and let $B_{l,g} \in \mathbb{R}^{d \times m}$ be the column block of B_l. The relative matrix-gradient scales are

$$p^{\mathrm{in}}_{l,g} = \frac{\operatorname{rms}_{\epsilon_p}(\nabla A_{l,g})}{\operatorname{rms}_{\epsilon_p}(A_{l,g})}, \qquad p^{\mathrm{out}}_{l,g} = \frac{\operatorname{rms}_{\epsilon_p}(\nabla B_{l,g})}{\operatorname{rms}_{\epsilon_p}(B_{l,g})}. \tag{11}$$

The coefficient-gradient energy scale for the global response is

$$p^{R}_{l,g} = \left[\frac{1}{2} \left(\operatorname{mean}((\nabla a_{l,g,\cdot})^2) + \operatorname{mean}((\nabla b_{l,g,\cdot})^2) \right) + \epsilon_p \right]^{1/2}. \tag{12}$$

MatrixPolicy maintains exponential moving averages $q^{\mathrm{in}}_{l,g}$, $q^{\mathrm{out}}_{l,g}$, and $q^{R}_{l,g}$ of these quantities with decay 0.95, and forms

$$\pi_{l,g} = \log q^{\mathrm{in}}_{l,g} - \log q^{\mathrm{out}}_{l,g}, \qquad \alpha_{l,g} = \log q^{R}_{l,g} - \frac{1}{2} \left(\log q^{\mathrm{in}}_{l,g} + \log q^{\mathrm{out}}_{l,g} \right). \tag{13}$$

Here each matrix pressure is an unsigned RMS relative-gradient scale. Consequently $\pi_{l,g}$ is a signed log imbalance between two unsigned magnitudes: its sign records which role currently has the larger RMS-relative scale, not a signed derivative of the loss along the positive-rescaling direction. MatrixPolicy uses that direction heuristically. The score $\alpha_{l,g}$ is a heuristic coefficient-activity statistic: because $p^{R}_{l,g}$ is an average of numerator- and denominator-gradient energies rather than a dimensionless matrix pressure, MatrixPolicy uses $\alpha_{l,g}$ only inside bounded redistribution and gating rules.

The activation also provides batch-estimated curve gains. From a recent sample of normalized coordinates $u_{l,g}$ cached by the RLB forward path, MatrixPolicy uses floored summaries

$$\hat{d}^{\text{live}}_{l,g} = \sqrt{\text{mean}\Big(R'_{l,g}(u_{l,g})^2\Big) + \epsilon_g}, \qquad \hat{o}^{\text{live}}_{l,g} = \sqrt{\text{mean}(R_{l,g}(u_{l,g})^2) + \epsilon_g}. \tag{14}$$

In equation 14, the mean is over all cached scalar normalized coordinates for group (l, g) across batch, sequence positions, and the m group coordinates. The derivative gain is a proxy for the input-domain role. The output gain is a curve-response proxy for the recombination role; it does not include the group radius $r_{l,g}$ and therefore is not the full feature scale seen by B_l. Both gains are deliberately cheap summaries rather than full block-Jacobian preconditioners.

3.3 MatrixPolicy Update Rule

Figure 2 summarizes the selective route before we give the equations: only the RLB matrix pair enters the MatrixPolicy chamber, while rational coefficients and ordinary Transformer parameters stay on an AdamW-only path.

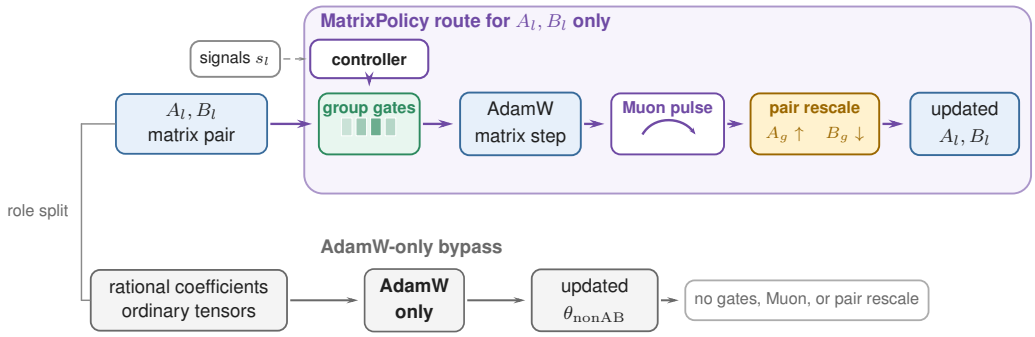

Figure 2: Selective MatrixPolicy route for the reported optimizer. Only the RLB matrix pair A_l, B_l enters the MatrixPolicy chamber: policy signals set group gradient gates $c_{l,g,\sigma}$, AdamW supplies the base matrix step, a transient Muon correction acts on the same matrices, and pair rescaling applies reciprocal updates $A_g \leftarrow e^{\ell} A_g$ and $B_g \leftarrow e^{-\ell} B_g$ to move the current balance γ toward the target τ. Rational coefficients and all non-RLB matrix parameters bypass this route and remain AdamW-only.

Let t be the optimizer step, T the planned number of training steps, and $\xi_t = t/T$. A smooth gate

$$\phi_t = \text{smoothstep}(0.02, 0.30, \xi_t) \tag{15}$$

turns on the group-gradient policy. For positive group values v_j, define $\text{geomean}^G_{j=1}(v_j) = \exp\Big(G^{-1} \sum_{j=1}^{G} \log v_j\Big)$. The multiplier for group g and role $\sigma \in \{\text{in}, \text{out}\}$ is

$$c_{l,g,\sigma} = c^{\text{gain}}_{l,g,\sigma} c^{\text{pressure}}_{l,g,\sigma} c^{\text{activity}}_{l,g}. \tag{16}$$

For the input role, the gain term uses $\hat{d}^{\text{live}}_{l,g}$; for the output role, it uses $\hat{o}^{\text{live}}_{l,g}$:

$$c^{\text{gain}}_{l,g,\text{in}} = \left(\frac{\text{geomean}^G_{j=1} \hat{d}^{\text{live}}_{l,j}}{\hat{d}^{\text{live}}_{l,g}} \right)^{0.20\phi_t}, \qquad c^{\text{gain}}_{l,g,\text{out}} = \left(\frac{\text{geomean}^G_{j=1} \hat{o}^{\text{live}}_{l,j}}{\hat{o}^{\text{live}}_{l,g}} \right)^{0.20\phi_t}. \tag{17}$$

With $\tilde{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.50, 1.50)$, the relative-gradient term sets role-dependent group multipliers before the later role-wise recentering:

$$c^{\text{pressure}}_{l,g,\text{in}} = \exp(-0.10\phi_t \tilde{\pi}_{l,g}), \qquad c^{\text{pressure}}_{l,g,\text{out}} = \exp(+0.10\phi_t \tilde{\pi}_{l,g}). \tag{18}$$

Coefficient-gradient activity downweights groups whose rational coefficients are moving more strongly than the adjacent matrices before role-wise recentering:

$$c^{\text{activity}}_{l,g} = \exp\left[-0.20\phi_t \, \text{ReLU}\left(\frac{\alpha_{l,g} - 0.05}{0.45} \right) \right]. \tag{19}$$

The final multiplier is recentered and clipped,

$$c_{l,g,\sigma} \leftarrow \frac{c_{l,g,\sigma}}{\mathrm{geomean}_{j=1}^{G} c_{l,j,\sigma}}, \qquad c_{l,g,\sigma} \leftarrow \mathrm{clip}(c_{l,g,\sigma}, 0.75, 1.35), \tag{20}$$

so the policy redistributes update scale across groups, separately for each matrix role, without changing the role-wise mean log multiplier before clipping.

The scaled matrix gradients are

$$\widetilde{\nabla} A_{l,g} = c_{l,g,\mathrm{in}} \nabla A_{l,g}, \qquad \widetilde{\nabla} B_{l,g} = c_{l,g,\mathrm{out}} \nabla B_{l,g}. \tag{21}$$

After this scaling, the RLB matrices are updated by a role/depth AdamW branch and a transient Muon branch. Define $\delta_l = (l-1)/(L-1)$ for $L > 1$ and $\delta_l = 1/2$ otherwise. The role factors are

$$\varrho_{\mathrm{in}}(l) = \mathrm{clip}(1 - 0.50(\delta_l - 0.5), 0.55, 1.40), \qquad \varrho_{\mathrm{out}}(l) = \mathrm{clip}(1 + 1.00(\delta_l - 0.5), 0.55, 1.40), \tag{22}$$

and the AdamW matrix multiplier for role σ is

$$s_{l,\sigma}^{\mathrm{Adam}} = \mathrm{clip}\left(3.0 \max\{0.10, 1 + 1.20(\varrho_\sigma(l) - 1)\}, 0.40, 4.0\right). \tag{23}$$

The Muon branch is concentrated early in training and is reduced when relative scale imbalance or coefficient-gradient scale is high. Define

$$\Psi_{l,t} = \mathrm{clip}\left(\frac{1}{G}\sum_{g=1}^{G} |\pi_{l,g}|, 0, 1.50\right), \qquad \Omega_{l,t} = \mathrm{ReLU}\left(\frac{G^{-1}\sum_{g=1}^{G} \alpha_{l,g} - 0.05}{0.45}\right), \tag{24}$$

$$\psi_{l,t} = \mathrm{clip}\left(\exp[-0.30\Psi_{l,t} - 0.65\Omega_{l,t}], 0.10, 1.15\right). \tag{25}$$

The branch gate is

$$\mu_{l,\sigma,t} = \mathrm{clip}\left(0.75\ \mathrm{smoothstep}(0.02, 0.12, \xi_t)[1 - \mathrm{smoothstep}(0.20, 0.36, \xi_t)]\varrho_\sigma(l)\psi_{l,t}, 0, 0.75\right). \tag{26}$$

For $M_{l,\mathrm{in}} = A_l$ and $M_{l,\mathrm{out}} = B_l$, MatrixPolicy applies an AdamW matrix step followed by any active Muon substep. The gate $\mu_{l,\sigma,t}$ scales the two learning rates; it is not a convex mixing coefficient for a single update direction:

$$\begin{aligned} \widetilde{M}_{l,\sigma}^{t+1} &= \mathrm{AdamW}_{\eta_t s_{l,\sigma}^{\mathrm{Adam}}(1-\mu_{l,\sigma,t})}(M_{l,\sigma}^t; \widetilde{\nabla} M_{l,\sigma}^t), \\ M_{l,\sigma}^{t+1} &= \mathrm{Muon}_{\eta_t \mu_{l,\sigma,t}}(\widetilde{M}_{l,\sigma}^{t+1}; \widetilde{\nabla} M_{l,\sigma}^t), \end{aligned} \tag{27}$$

Equation equation 27 is operational update notation: AdamW and Muon carry separate optimizer states, their configured weight-decay and momentum terms are handled by the respective optimizer implementations, and both substeps use the gradient from training step t. Here η_t is the base learning rate and Muon denotes the `torch.optim.Muon` update on two-dimensional RLB matrix parameters: momentum 0.95, five Newton–Schulz orthogonalization steps, and the match-RMS learning-rate adjustment of the Muon optimizer (Liu et al., 2025). When all Muon fractions have decayed to zero, this branch is inactive.

The last operation is a bounded positive rescaling of the two matrix blocks around each rational curve. It is evaluated every five optimizer steps; on other steps it is skipped. Let $\mathcal{U} = \{u_k\}_{k=1}^{129}$ be a uniform probe grid over $[-5, 5]$, and define $\mathrm{rms}_{\mathcal{U}}(\varphi) = \sqrt{|\mathcal{U}|^{-1}\sum_{u_k \in \mathcal{U}} \varphi(u_k)^2}$. The fixed-grid probe gains are diagnostics of the curve shape, not estimates of the current feature distribution. They are used only by the pair-rescaling heuristic, while the live gains in equation 14 provide the batch-distribution summaries for group-gradient redistribution. The probe gains are

$$\hat{d}_{l,g}^{\mathrm{probe}} = \sqrt{|\mathcal{U}|^{-1}\sum_{u_k \in \mathcal{U}} R'_{l,g}(u_k)^2 + \epsilon_g}, \qquad \hat{o}_{l,g}^{\mathrm{probe}} = \sqrt{|\mathcal{U}|^{-1}\sum_{u_k \in \mathcal{U}} R_{l,g}(u_k)^2 + \epsilon_g}, \tag{28}$$

and the current matrix log ratio is computed with floored matrix RMS values,

$$\gamma_{l,g} = \log \mathrm{rms}_{\epsilon_p}(A_{l,g}) - \log \mathrm{rms}_{\epsilon_p}(B_{l,g}). \tag{29}$$

Because the floor is inside both logarithms, $\gamma_{l,g}$ is a stable implementation proxy for a matrix-RMS ratio; near the floor it is not an exact log-norm ratio. The heuristic target ratio combines curve-gain and relative-gradient terms,

$$\tau_{l,g} = \frac{1}{2}\left(\log \hat{d}_{l,g}^{\text{probe}} - \log \hat{o}_{l,g}^{\text{probe}}\right) + 0.25\,\bar{\pi}_{l,g}, \qquad \bar{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.25, 1.25). \tag{30}$$

The sign and factors in $\tau_{l,g}$ are design heuristics rather than a derived optimality condition. The first term balances fixed-grid derivative and response gains in log space, with the factor $1/2$ mapping the gain contrast to the matrix-pair log-ratio coordinate; the second term adds a smaller pressure bias from the clipped RMS-imbalance proxy. Coefficient-gradient scale modulates the rescaling strength through

$$a_{l,g}^{\text{rs}} = \exp(0.10\,\bar{\alpha}_{l,g}), \qquad \bar{\alpha}_{l,g} = \text{clip}(\alpha_{l,g}, -1.25, 1.25). \tag{31}$$

The positive sign is intentional: high coefficient activity damps the group-gradient and Muon paths, but lets the bounded balancing step move slightly more quickly toward the representative ratio; the later log-step clip limits this effect. With schedule

$$s_{l,t} = 0.50\ \text{smoothstep}(0.03, 0.35, \xi_t)\,\text{clip}(1 + 0.15(\delta_l - 0.5), 0.75, 1.25), \tag{32}$$

the active-step log move is

$$\ell_{l,g,t} = \text{clip}\left(\frac{1}{2}[\tau_{l,g} - \gamma_{l,g}]s_{l,t}a_{l,g}^{\text{rs}}, -0.030, 0.030\right). \tag{33}$$

The matrix blocks are then updated as

$$A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}, \qquad B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}. \tag{34}$$

The implementation applies the corresponding group scale to optimizer-state buffers as well, using squared scales for square-average state entries.

4 Experiments

4.1 Fixed-Scale Target-Arrival Study

The fixed-scale study uses the same 12-layer, width-768 causal Transformer on five corpora: DCLM/DataComp-LM (Li et al., 2024), FineWeb-Edu and FineWeb (Penedo et al., 2024), Dolma-sample (Soldaini et al., 2024), and C4 (Raffel et al., 2020). The 100M-token budget uses 3050 optimizer steps and the 300M-token budget uses 9150 optimizer steps, with three seeds per method/dataset pair. The main comparison keeps the architecture and data fixed while changing the activation and optimizer used in the nonlinear sublayer: MatrixPolicy is compared with SiLU controls and with RLB-only controls, where RLB-only uses the same global-rational, no-local-atom activation but no MatrixPolicy updates. Shared optimizer, evaluation-cadence, and throughput-measurement details are reported in Appendix A; broader optimizer comparisons beyond AdamW and Muon are summarized in Table 2.

Table 1: Target-arrival efficiency for the fixed 12-layer, width-768 Transformer. RLB denotes the global-rational, no-local-atom activation without MatrixPolicy. At the hardest validation-loss target reached by every listed method in all three seeds, MatrixPolicy is always fastest; each token-saving column reports MatrixPolicy's savings relative to the method named below it.

			MatrixPolicy token savings (%)				Time to target (min)				
Budget	Dataset	Target loss	vs. SiLU AdamW	vs. SiLU Muon	vs. RLB AdamW	vs. RLB Muon	Matrix Policy	SiLU AdamW	SiLU Muon	RLB AdamW	RLB Muon
100M	DCLM	4.55	20.7	32.4	20.7	34.3	12.8	20.8	26.0	14.3	19.0
	FineWeb-Edu	4.40	19.5	29.5	20.2	29.5	12.9	20.3	24.4	14.2	18.0
	FineWeb	4.60	22.8	32.1	21.5	32.1	12.6	16.4	20.2	16.2	20.1
	Dolma	4.60	22.0	34.0	22.7	34.9	12.8	17.6	22.0	17.3	22.2
	C4	4.60	20.7	29.2	19.3	28.7	11.6	14.3	17.3	14.9	18.8
300M	DCLM	4.10	23.3	8.3	24.0	7.0	39.6	53.5	45.9	54.7	45.6
	FineWeb-Edu	3.85	22.8	6.7	22.6	4.8	40.0	59.9	47.9	52.3	50.0
	FineWeb	4.10	23.1	7.1	22.4	5.3	40.8	61.2	46.4	55.6	46.6
	Dolma	3.95	22.3	9.8	22.3	6.5	44.4	49.3	52.3	50.5	47.4
	C4	4.00	26.4	8.5	25.3	6.5	41.8	64.7	55.5	59.4	51.9

Table 1 is the primary empirical result. It asks: for the hardest common validation-loss target reached by every listed method in all three seeds, what fraction of target-arrival tokens does MatrixPolicy save, and how many clean target-arrival minutes does each method require? Across all five datasets under the 100M-token budget, MatrixPolicy saves 19.5–22.8% of the SiLU with AdamW target-arrival tokens and 29.2–34.0% of the SiLU with Muon target-arrival tokens. Under the 300M-token budget, after longer training compresses endpoint gaps, MatrixPolicy still saves 22.3–26.4% versus SiLU with AdamW and 6.7–9.8% versus SiLU with Muon, with lower target-arrival time in every listed comparison.

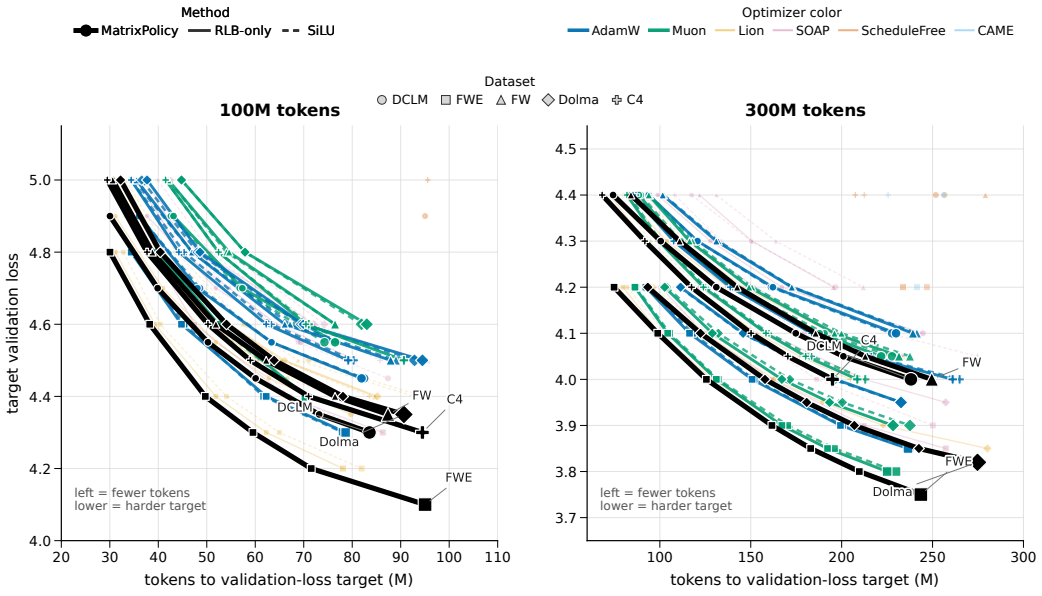

Figure 3: Target-arrival map for the completed fixed-scale study. Each point is the mean token count needed by one method to reach one validation-loss target on one dataset; connected points trace the same method and dataset from easier to harder targets. The horizontal coordinate is target-arrival tokens, and the vertical coordinate is the target validation loss, so curves further left reach the same loss with fewer tokens. The hard-target timing and token-saving percentages are quantified in Table 1.

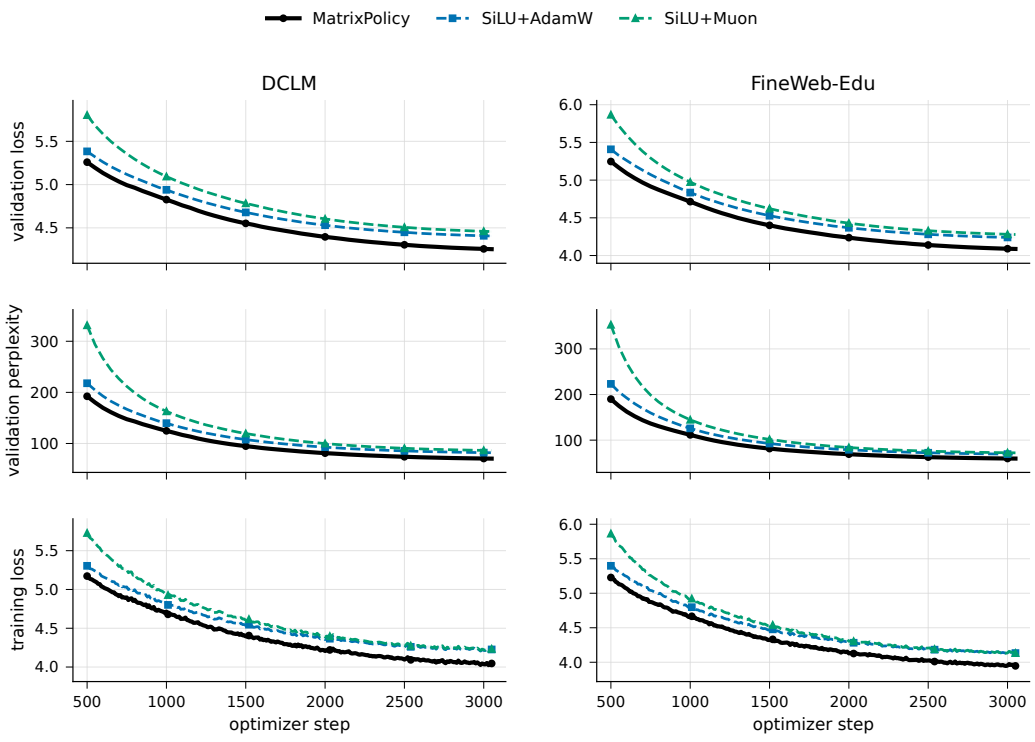

Figure 4: Representative 100M-token-budget training dynamics on DCLM and FineWeb-Edu. The rows show validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation. This figure uses the two standard SiLU baselines so that the early target-arrival effect in Table 1 remains visually legible.

The 100M-token-budget curves show why target arrival is the right main readout. MatrixPolicy separates early in validation loss and validation perplexity while keeping training loss competitive. A long fixed budget eventually compresses many activation/optimizer endpoints, especially under the 300M-token budget, so the paper emphasizes the number of tokens and minutes needed to reach the same useful validation-loss levels rather than only the final loss after every method has consumed the full budget.

4.2 TODO: Larger-Scale Training Results

TODO: This subsection is reserved for larger-scale training results. The current claim is limited to the completed fixed-scale evidence in Section 4.1, Table 1, and Figures 3–4.

4.3 TODO: Component Ablations

TODO: This subsection is reserved for component ablations isolating the centered group redistribution, transient Muon gate, bounded pair rescaling, and the global rational activation interface.

5 Conclusion

Global-rational RLB turns the nonlinear Transformer sublayer into an optimizer-visible interface: it exposes grouped normalized coordinates, trainable global rational responses, and an idealized positive matrix-pair relation used as a bounded balancing heuristic. MatrixPolicy uses that interface to update the RLB input and output matrices with role-aware statistics, while rational coefficients and non-RLB-matrix parameters use AdamW. The completed 100M-token and 300M-token runs show a consistent target-arrival pattern: MatrixPolicy reaches common validation losses with fewer tokens

and lower measured target-arrival time than the listed SiLU and global-rational controls. Larger-scale training and component ablations remain explicit TODO subsections inside the Experiments section of the current draft.

REFERENCES

Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Lu, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic Discovery of Optimization Algorithms. In *Advances in Neural Information Processing Systems*, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.

Aaron Defazio, Xingyu Yang, Ahmed Khaled, Konstantin Mishchenko, Harsh Mehta, and Ashok Cutkosky. The Road Less Scheduled. In *Advances in Neural Information Processing Systems*, 2024. URL https://neurips.cc/virtual/2024/poster/96925.

Dan Hendrycks and Kevin Gimpel. Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. URL https://arxiv.org/abs/1606.08415.

Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015. URL https://mlanthology.org/iclr/2015/kingma2015iclr-adam/.

Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, et al. DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0455. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets_and_Benchmarks_Track.html.

Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious Optimizers: Improving Training with One Line of Code. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=zBPZeRjfgu.

Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/06960915ba8674c7a898ec0b472b80ff-Abstract-Conference.html.

Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is Scalable for LLM Training. *arXiv preprint arXiv:2502.16982*, 2025. URL https://arxiv.org/abs/2502.16982.

Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

Yang Luo, Xiaozhe Ren, Zangwei Zheng, Zhuo Jiang, Xin Jiang, and Yang You. CAME: Confidence-guided Adaptive Memory Efficient Optimization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pp. 4442–4453, 2023. doi: 10.18653/v1/2023.acl-long.243. URL https://aclanthology.org/2023.acl-long.243/.

Guilherme Penedo, Hynek Kydlicek, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0970. URL https://papers.nips.cc/paper/2024/

hash/370df50ccfdf8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL http://jmlr.org/papers/v21/20-074.html.

Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for Activation Functions. *arXiv preprint arXiv:1710.05941*, 2017. URL https://arxiv.org/abs/1710.05941.

Noam Shazeer. GLU Variants Improve Transformer. *arXiv preprint arXiv:2002.05202*, 2020. URL https://arxiv.org/abs/2002.05202.

Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, et al. Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. *arXiv preprint arXiv:2402.00159*, 2024. URL https://arxiv.org/abs/2402.00159.

Maosen Tang and Alex Townsend. Rational Neural Networks have Expressivity Advantages. *arXiv preprint arXiv:2602.12390*, 2026. doi: 10.48550/arXiv.2602.12390. URL https://arxiv.org/abs/2602.12390.

Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/e988664070e9591f93fdcf605f7dc623-Abstract-Conference.html.

Kaiyue Wen, David Leo Wright Hall, Tengyu Ma, and Percy Liang. Fantastic Pretraining Optimizers and Where to Find Them. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=2J51qUZ0iG.

Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Diederik P. Kingma, Yinyu Ye, Zhi-Quan Luo, and Ruoyu Sun. Adam-mini: Use Fewer Learning Rates To Gain More. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/45ae878717399e6f62d57c65f052cd46-Abstract-Conference.html.

Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. In *International Conference on Machine Learning*, 2024. URL https://openreview.net/forum?id=hYHsrKDiX7.

A Experimental Details and Supporting Results

A.1 Model and data pipeline

The fixed-scale experiments use a custom causal decoder Transformer rather than an off-the-shelf GPT-2 or LLaMA implementation. The block uses a LLaMA-style pre-normalized layout with RMSNorm before attention and before the nonlinear sublayer, bias-free query/key/value and output attention projections, rotary position embeddings, causal scaled-dot-product attention, residual attention and nonlinear-sublayer updates, a final RMSNorm, and a tied token-embedding/output head. The fixed model has 12 layers, width 768, 12 heads of dimension 64, and context length 256.

The SiLU controls use a SwiGLU-like gated nonlinear sublayer with bias-free matrices $W_g, W_v \in \mathbb{R}^{768\times 2048}$ and $W_o \in \mathbb{R}^{2048\times 768}$, computing $W_o(\mathrm{SiLU}(xW_g) \odot xW_v)$. The global-rational RLB rows use a matched single-branch matrix pair: an input matrix maps width 768 to 3072 latent coordinates, the coordinates are partitioned into groups and transformed by the real-pole-free P5/Q4 global rational response, and an output matrix maps the 3072 coordinates back to width 768. This is the no-local-atom RLB variant used by the main MatrixPolicy results.

Tokenization uses the GPT-2 tokenizer through the Hugging Face tokenizer interface, without adding tokenizer-specific special tokens; an EOS or pad token is appended between documents. Training samples random contiguous blocks of length 257 and forms next-token prediction pairs of length 256. The DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4 runs use the text field specified in the manifests, with validation slices drawn from held-out token offsets of the same corpus stream.

A.2 Training protocol

All runs use batch size 16, gradient accumulation 2, and therefore 32768 global tokens per optimizer step. The 100M-token budget consists of 3050 optimizer steps; the 300M-token budget consists of 9150 optimizer steps. Each method/dataset pair is run with seeds 1337, 2027, and 3407. Validation is performed every 50 optimizer steps, so token-to-target arrivals are quantized in increments of 1.64M tokens.

The MatrixPolicy rows in the main table use the global-rational RLB activation with no local atoms: the active nonlinearity is only the group-global P5/Q4 response in equation 6–equation 7. The reported configuration uses AdamW as the backbone optimizer and disables the separate rational-coefficient optimizer. It enables MatrixPolicy group redistribution with gain/pressure/activity strengths $0.20/0.10/0.20$, schedule window 0.02–0.30, and group multiplier clip $[0.75, 1.35]$. The bounded RLB pair-rescale wrapper runs every five optimizer steps with covariant optimizer state scaling. Rational coefficients, attention weights, embeddings, normalization parameters, and ordinary Transformer weights use AdamW in this configuration.

Target-arrival minutes in Table 1 are computed seed-wise from the first validation event that reaches the shared target and the same run's clean measured seconds per optimizer step. They are not computed from queue time or whole-job elapsed time across restarts.

A.3 SUPPORTING FIXED-SCALE CURVES

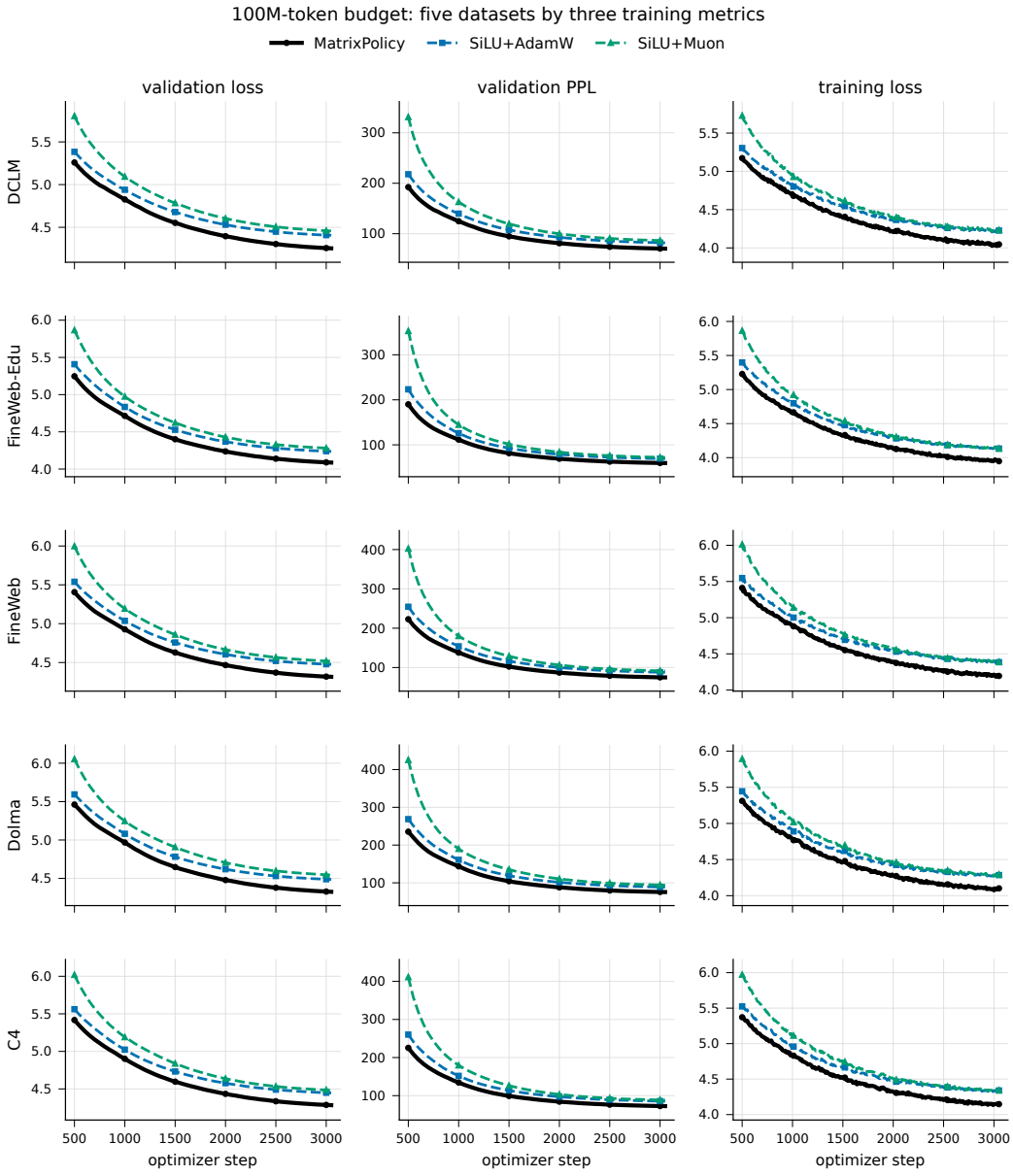

Figure 5: 100M-token-budget dynamics across all five datasets. Rows are datasets and columns are validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation.

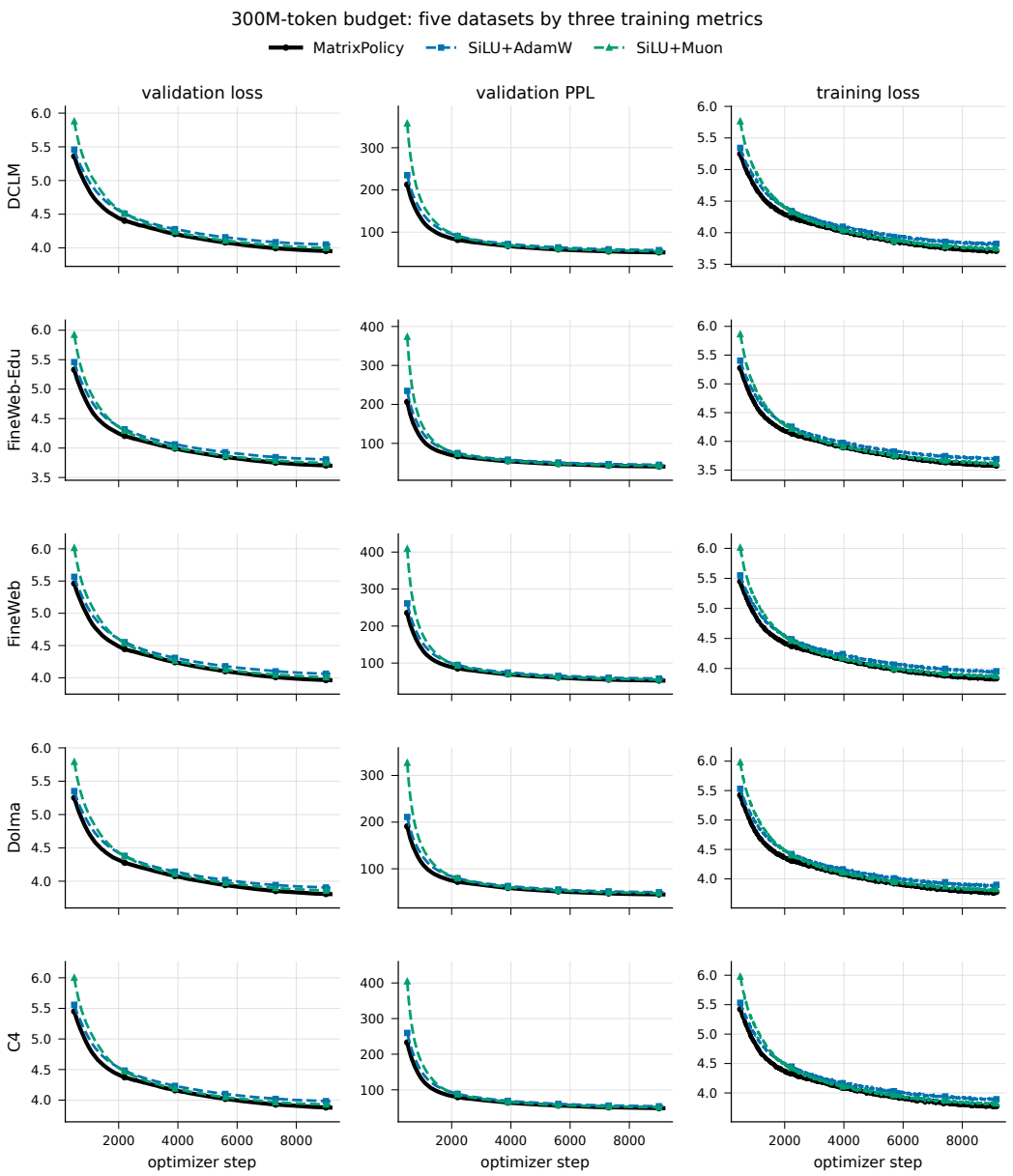

Figure 6: 300M-token-budget dynamics across all five datasets. Rows are datasets and columns are validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation.

Table 2: Final validation loss for the broader optimizer sweep on the fixed 12-layer, width-768 language model. Values are mean ± sample standard deviation over three seeds; lower is better. RLB rows use the global-rational, no-local-atom activation without MatrixPolicy.

Budget	Method	DCLM	FineWeb-Edu	FineWeb	Dolma	C4
100M tokens	MatrixPolicy	4.2538±0.0063	4.0873±0.0102	4.3162±0.0125	4.3253±0.0053	4.2837±0.0197
	SiLU + AdamW	4.4056±0.0099	4.2375±0.0086	4.4758±0.0097	4.4862±0.0012	4.4469±0.0160
	SiLU + Lion	4.3183±0.0069	4.1494±0.0092	4.3825±0.0083	4.3878±0.0046	4.3536±0.0156
	SiLU + Muon	4.4572±0.0126	4.2787±0.0243	4.5163±0.0264	4.5443±0.0108	4.4824±0.0233
	SiLU + SOAP	4.4160±0.0038	4.2630±0.0220	4.4840±0.0112	4.4987±0.0035	4.4582±0.0146
	SiLU + ScheduleFree	4.9023±0.0111	4.8258±0.0072	5.0142±0.0188	5.0494±0.0149	5.0064±0.0160
	SiLU + CAME	5.0107±0.0143	4.9207±0.0123	5.1325±0.0126	5.1650±0.0189	5.1230±0.0178
	RLB + AdamW	4.4046±0.0081	4.2392±0.0077	4.4717±0.0138	4.4878±0.0027	4.4412±0.0162
	RLB + Lion	4.2946±0.0083	4.1361±0.0083	4.3626±0.0112	4.3622±0.0066	4.3271±0.0160
	RLB + Muon	4.4674±0.0121	4.2831±0.0284	4.5157±0.0059	4.5476±0.0153	4.4764±0.0099
	RLB + SOAP	4.4360±0.0442	4.2747±0.0238	4.6893±0.3359	4.5183±0.0363	4.4458±0.0190
	RLB + ScheduleFree	4.8871±0.0046	4.7957±0.0133	4.9993±0.0192	5.0363±0.0129	4.9842±0.0164
	RLB + CAME	5.0139±0.0086	4.9150±0.0063	5.1340±0.0162	5.1514±0.0168	5.1109±0.0155
300M tokens	MatrixPolicy	3.9518±0.0282	3.7015±0.0212	3.9623±0.0081	3.8062±0.0073	3.8777±0.0144
	SiLU + AdamW	4.0493±0.0275	3.8035±0.0182	4.0612±0.0101	3.9037±0.0091	3.9811±0.0128
	SiLU + Lion	3.9934±0.0230	3.7440±0.0208	4.0015±0.0085	3.8475±0.0094	3.9213±0.0105
	SiLU + Muon	3.9973±0.0305	3.7454±0.0170	4.0066±0.0128	3.8581±0.0101	3.9251±0.0134
	SiLU + SOAP	4.0964±0.0300	3.8629±0.0201	4.1139±0.0101	3.9568±0.0093	4.0349±0.0108
	SiLU + ScheduleFree	4.3657±0.0298	4.1559±0.0238	4.3979±0.0106	4.2151±0.0053	4.3163±0.0107
	SiLU + CAME	4.3682±0.0226	4.1503±0.0211	4.4060±0.0189	4.2492±0.0270	4.3298±0.0148
	RLB + AdamW	4.0496±0.0298	3.8024±0.0206	4.0601±0.0110	3.9045±0.0082	3.9787±0.0098
	RLB + Lion	3.9887±0.0295	3.7416±0.0214	3.9960±0.0105	3.8412±0.0085	3.9132±0.0139
	RLB + Muon	3.9913±0.0260	3.7373±0.0187	3.9991±0.0110	3.8478±0.0047	3.9189±0.0149
	RLB + SOAP	4.0608±0.0331	3.8240±0.0128	4.1081±0.0320	3.9269±0.0125	4.0024±0.0194
	RLB + ScheduleFree	4.3607±0.0344	4.1421±0.0217	4.3864±0.0081	4.2121±0.0112	4.3084±0.0071
	RLB + CAME	4.4503±0.0426	4.2255±0.0358	4.4804±0.0064	4.2665±0.0409	4.3651±0.0582

B RLB Calculus and Matrix-Role Structure

B.1 Real-pole-free group-global response

For one layer and group, write

$$P(u) = \sum_{i=0}^{5} a_i u^i, \qquad Q(u) = 1 + \sum_{j=1}^{4} |b_j| \phi_j(u), \qquad \phi(u) = (|u|, u^2, |u|^3, u^4). \tag{35}$$

Since $\phi_j(u) \geq 0$ for all real u, $Q(u) \geq 1$ and $R(u) = P(u)/Q(u)$ has no real pole. Away from $u = 0$ for input derivatives,

$$R'(u) = \frac{P'(u)Q(u) - P(u)Q'(u)}{Q(u)^2}, \tag{36}$$

where

$$P'(u) = \sum_{i=1}^{5} i a_i u^{i-1}, \qquad Q'(u) = |b_1| \operatorname{sign}(u) + 2|b_2|u + 3|b_3|u|u| + 4|b_4|u^3. \tag{37}$$

The implementation uses the autodiff convention $\operatorname{sign}(0) = 0$ for derivative-gain statistics. The coefficient features are explicit:

$$\frac{\partial R(u)}{\partial a_i} = \frac{u^i}{Q(u)}, \qquad \frac{\partial R(u)}{\partial b_j} = -\frac{P(u)}{Q(u)^2} \operatorname{sign}(b_j) \phi_j(u), \tag{38}$$

with the usual autodiff subgradient convention at $b_j = 0$. These derivatives make output-factor and derivative gains directly observable from the layer/group global curve, without introducing local-atom parameters.

B.2 Normalized-group Jacobian and gain summaries

For a group vector $z \in \mathbb{R}^m$, define

$$r(z) = \sqrt{m^{-1}\|z\|_2^2 + \epsilon_{\mathrm{rms}}}, \qquad u(z) = z/r(z), \qquad h(z) = r(z)R(u(z)), \tag{39}$$

where R is applied coordinatewise. The radius derivative is

$$\mathrm{d}r = \frac{z^\top \mathrm{d}z}{mr} = \frac{u^\top \mathrm{d}z}{m}, \tag{40}$$

and the normalized-coordinate derivative is

$$\mathrm{d}u = \frac{1}{r}\left(I - \frac{uu^\top}{m}\right)\mathrm{d}z. \tag{41}$$

Therefore the group Jacobian is

$$J_h(z) := \frac{\partial h}{\partial z} = R(u)\frac{u^\top}{m} + \mathrm{diag}(R'(u))\left(I - \frac{uu^\top}{m}\right). \tag{42}$$

This decomposition motivates separate output-gain and derivative-gain summaries. The vector $R(u)$ controls the curve-response factor inside the realized feature $h = rR(u)$, while $R'(u)$ appears in the tangent component of the input Jacobian seen by A_l. The summaries $\hat{o}$ and $\hat{d}$ are not full feature-scale or input-Jacobian measurements: the full feature also depends on r, and the full input gradient also depends on the radial $R(u)u^\top/m$ term and on $B_{l,g}$.

B.3 Positive matrix-pair rescaling with an RMS floor

Set $\epsilon_{\mathrm{rms}} = 0$ and take $a > 0$. Then

$$r(az) = ar(z), \qquad u(az) = u(z), \qquad h(az) = ah(z). \tag{43}$$

Scaling $A_{l,g}$ by a and the matching columns of $B_{l,g}$ by a^{-1} therefore leaves the group contribution unchanged. Applying this independently to all groups gives equation 9.

With $\epsilon_{\mathrm{rms}} > 0$, define the floored maps

$$r_\epsilon(z) = \sqrt{m^{-1}\|z\|_2^2 + \epsilon_{\mathrm{rms}}}, \qquad u_\epsilon(z) = z/r_\epsilon(z), \qquad h_\epsilon(z) = r_\epsilon(z)R(u_\epsilon(z)). \tag{44}$$

For $\rho^2 = m^{-1}\|z\|_2^2$,

$$u_\epsilon(az) = \kappa(a,z)u_\epsilon(z), \qquad \kappa(a,z) = \frac{a\sqrt{\rho^2 + \epsilon_{\mathrm{rms}}}}{\sqrt{a^2\rho^2 + \epsilon_{\mathrm{rms}}}}. \tag{45}$$

Let L_R be a Lipschitz constant of the coordinatewise curve on the segment between $u_\epsilon(z)$ and $\kappa(a,z)u_\epsilon(z)$. After compensating the output matrix by a^{-1}, the group-feature error obeys

$$\left\|a^{-1}h_\epsilon(az) - h_\epsilon(z)\right\|_2 \le \left|\frac{r_\epsilon(az)}{a} - r_\epsilon(z)\right| \left\|R(u_\epsilon(z))\right\|_2 \tag{46}$$

$$+ \frac{r_\epsilon(az)}{a} L_R |\kappa(a,z) - 1| \left\|u_\epsilon(z)\right\|_2. \tag{47}$$

Both terms vanish when $\epsilon_{\mathrm{rms}} = 0$. When $\rho^2 \gg \epsilon_{\mathrm{rms}}$ and the global response and derivative are bounded on the segment between $u_\epsilon(z)$ and $\kappa(a,z)u_\epsilon(z)$, the bound is small; near the floor, the approximation can be poor. This is the reason MatrixPolicy uses clipped rescaling moves rather than treating matrix-pair balancing as an exact function-preserving operation.

C MatrixPolicy Update Details and Design Rationale

C.1 Optimizer step order

At training step t, backpropagation produces gradients with respect to the pre-update parameters. MatrixPolicy computes the pressure and activity statistics from these gradients and from A_l^t, B_l^t,

and the rational coefficients before any parameter is changed. With EMA decay $\beta = 0.95$, the matrix and coefficient statistics have the form

$$q_{l,g}^{\mathrm{in,t}} = \beta q_{l,g}^{\mathrm{in,t-1}} + (1-\beta) p_{l,g}^{\mathrm{in,t}}, \tag{48}$$

$$q_{l,g}^{\mathrm{out,t}} = \beta q_{l,g}^{\mathrm{out,t-1}} + (1-\beta) p_{l,g}^{\mathrm{out,t}}, \tag{49}$$

$$q_{l,g}^{R,t} = \beta q_{l,g}^{R,t-1} + (1-\beta) p_{l,g}^{R,t}. \tag{50}$$

The live curve gains are read from the RLB forward cache that produced the same loss, so the gates in equation 16–equation 26 use pre-update statistics for the current minibatch. Non-matrix Transformer parameters and rational coefficients then follow the AdamW path. The RLB matrix blocks use the scaled gradients in equation 21 for the AdamW and Muon matrix branches in equation 27. On scheduled rescaling steps, the bounded positive matrix-pair move in equation 34 is applied after the matrix branch, and the corresponding optimizer-state buffers are scaled covariantly. Thus MatrixPolicy changes only the representatives of RLB input/output matrix pairs; it does not introduce a separate optimizer path for rational coefficients or for non-RLB tensors.

C.2 Design rationale

This appendix explains why MatrixPolicy uses role-specific gains, centered relative-gradient redistribution, and bounded pair rescaling. The argument is not a derivation of an optimal optimizer. It identifies the matrix-role quantities exposed by RLB and states the approximations under which the policy rules are meaningful.

C.3 Role-specific gradients and gain selection

For one training example, let $\bar{y}_l = \nabla_{y_l}\mathcal{L}$ be the upstream gradient, and let $J_{l,g} = \partial h_{l,g}/\partial z_{l,g}$ be the group Jacobian in equation 42. The adjacent matrix gradients factor as

$$\nabla_{B_{l,g}}\mathcal{L} = \bar{y}_l h_{l,g}^\top, \tag{51}$$

$$\nabla_{A_{l,g}}\mathcal{L} = \left(J_{l,g}^\top B_{l,g}^\top \bar{y}_l\right) x_l^\top. \tag{52}$$

The output matrix is coupled directly to the realized feature $h_{l,g}$, whereas the input matrix is coupled to the tangent and radial components of the RLB Jacobian before the upstream gradient reaches x_l. This asymmetry motivates separate gain summaries: the response RMS $\hat{o}$ is an output-role proxy, and the derivative RMS $\hat{d}$ is an input-role proxy. These summaries are partial by construction. The full output feature also contains $r_{l,g}$, and the full input gradient also depends on $B_{l,g}$ and on the radial term $R(u)u^\top/m$ in equation 42. MatrixPolicy therefore uses gains only as bounded multipliers, not as full preconditioners.

C.4 Scale-normalized pressure and centered redistribution

In the idealized unfloored rescaling direction, an infinitesimal increase of the input representative and reciprocal decrease of the output representative has the loss derivative

$$\Delta_g = \left\langle \nabla_{A_{l,g}}\mathcal{L}, A_{l,g} \right\rangle_F - \left\langle \nabla_{B_{l,g}}\mathcal{L}, B_{l,g} \right\rangle_F. \tag{53}$$

MatrixPolicy does not estimate Δ_g directly. Instead it uses RMS relative gradient scales because they are cheap and stable. For nonzero unfloored W, if $W^+ = W - \eta\mathcal{G}$ and η is small, then

$$\log \mathrm{rms}(W^+) - \log \mathrm{rms}(W) = -\eta \frac{\langle \mathcal{G}, W\rangle_F}{\|W\|_F^2} + O(\eta^2), \tag{54}$$

and Cauchy–Schwarz implies

$$\left| \frac{\langle \mathcal{G}, W\rangle_F}{\|W\|_F^2} \right| \le \frac{\|\mathcal{G}\|_F}{\|W\|_F} = \frac{\mathrm{rms}(\mathcal{G})}{\mathrm{rms}(W)}. \tag{55}$$

The implemented quantities p^{in} and p^{out} are floored versions of the upper-bound scale on the right-hand side. They are unsigned magnitudes. Thus $\pi_{l,g} = \log q_{l,g}^{\mathrm{in}} - \log q_{l,g}^{\mathrm{out}}$ should be read as an RMS-imbalance proxy whose sign indicates which role has the larger relative scale. Using that sign

directionally in equation 18 and equation 30 is a policy heuristic, not a claim that $\pi_{l,g}$ is the signed derivative in equation 53.

The redistribution step is centered separately for each role. Before clipping,

$$\widehat{c}_{l,g,\sigma} = \frac{c_{l,g,\sigma}}{\mathrm{geomean}_{j=1}^{G} c_{l,j,\sigma}}, \qquad \frac{1}{G}\sum_{g=1}^{G} \log \widehat{c}_{l,g,\sigma} = 0. \tag{56}$$

The policy therefore redistributes update scale across groups while preserving the role-wise mean log multiplier before clipping. The clip in equation 20 deliberately trades exact centering for robustness to noisy group statistics.

C.5 Bounded matrix-pair balancing and transient direction updates

For the unfloored nonzero-matrix calculation, write

$$\gamma_{l,g} = \log \mathrm{rms}(A_{l,g}) - \log \mathrm{rms}(B_{l,g}). \tag{57}$$

An active rescaling step applies $A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}$ and $B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}$, hence

$$\gamma_{l,g}^{+} = \gamma_{l,g} + 2\ell_{l,g,t}. \tag{58}$$

If clipping is ignored and $\lambda_{l,g,t} = s_{l,t} a_{l,g}^{\mathrm{rs}}$, then equation 33 gives

$$\gamma_{l,g}^{+} = (1 - \lambda_{l,g,t})\gamma_{l,g} + \lambda_{l,g,t}\tau_{l,g}. \tag{59}$$

This is an exact contraction only in the nonzero-matrix, unfloored-RMS, $\epsilon_{\mathrm{rms}} = 0$, unclipped idealization. The implementation uses the floored proxy in equation 29; near the floor, the log-ratio interpretation and the contraction model are approximate.

The target

$$\tau_{l,g} = \frac{1}{2}\left(\log \hat{d}_{l,g}^{\mathrm{probe}} - \log \hat{o}_{l,g}^{\mathrm{probe}}\right) + 0.25\,\bar{\pi}_{l,g} \tag{60}$$

uses fixed-grid probe gains because pair rescaling is meant to balance the representative around the curve shape rather than chase minibatch feature noise. The sign and factors are heuristic: the derivative/response contrast supplies a log-space gain-balancing target, the factor $1/2$ maps that contrast to the matrix-pair ratio coordinate, and the smaller pressure term biases the target using the clipped RMS-imbalance proxy. These constants are not claimed to solve an optimal-control problem.

The transient Muon branch uses

$$w(\xi_t) = \mathrm{smoothstep}(0.02, 0.12, \xi_t)\,[1 - \mathrm{smoothstep}(0.20, 0.36, \xi_t)]\,. \tag{61}$$

The window confines the matrix-direction update to the phase in which changes to latent directions most directly affect the coordinates seen by the rational curves. The penalty $\psi_{l,t}$ reduces this branch when RMS imbalance or coefficient-gradient activity is high. MatrixPolicy therefore uses Muon as a bounded, state-dependent matrix-direction substep rather than as a persistent generic optimizer for all tensors.